

Architetrix

Project Specification | Team Architetrix

Team

Adrian Architect, Exhibitor, Builder, Facilitator

Džiugas Critic, Designer, Builder, Facilitator

Contents

1	Game Idea	1
1.1	Core Gameplay	2
1.2	Game Concept	2
1.2.1	Pieces	2
1.2.2	Use Case Diagram	3
1.2.3	Board	3
1.2.4	Core Mechanic	3
1.2.5	Clearing Rows / Columns	4
1.3	Difficulty Curve	4
1.4	Technology Stack	4
1.5	Activity Diagram	5
1.6	Data Flow Diagram	6
1.7	High-Level Architecture	7
1.8	UML diagrams	8
1.8.1	First Iteration	8
1.8.2	Second Iteration - after TP2	8
2	Plan for TP1 & TP2	9
3	File distribution for TP1 & TP2	10
4	Git Repository	10
5	Project Costs	10
6	Demo Video TP3	11

1. Game Idea

Architetrix is a Tetris-style console game built around bitwise operations.

Figure 1: Proof of Concept

Buffer	Architetrix	NEXT

111	0	1110
1 00	1011	1
	1	
		110
		001
Score:		
1212		10 1
		111
Lvl: 3		
	1	OR
	01 01	OR
	111 011	NAND
	1101101	OR
	1011110	AND
	10010101	XOR
	01001111	NAND

Link to Proof of Concept Gif: [Proof of Concept Gif](#)

1.1. Core Gameplay

Pieces made of 0s and 1s fall onto a fixed-width grid. Each row has a predefined bit pattern and a bitwise operator (**AND**, **OR**, **XOR**, etc.). When a piece touches a row, its bits combine with the row's bits using that operation - a 1 fills a cell, while a 0 clears it.

Bits continue falling after interactions, allowing chain reactions across multiple rows. Rows or columns fully filled with 1s are cleared for points, with higher rewards for more complex clears. New rows appear from the bottom over time, spawning faster as the game progresses. The goal is to survive as long as possible without the stack reaching the top while scoring as high as possible.

1.2. Game Concept

1.2.1 Pieces

Pieces are bit patterns (e.g., 11011, 010, L-shapes, U-shapes, ...).

1.2.2 Use Case Diagram

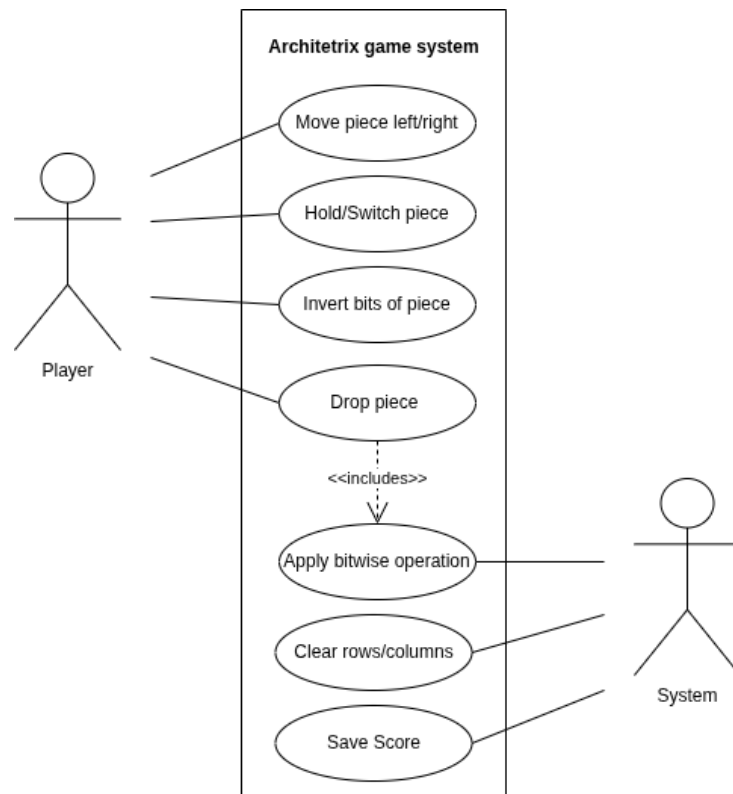


Figure 2: Use Case Diagram

Available operations:

- **Move** piece left/right.
- **Hold/Switch** - stores the piece in a buffer. If the buffer already holds a piece, they are swapped. Can only be used once per new piece.
- **Invert (NOT)** - flips all bits (010 → 101).
- **Drop** - the piece instantly reaches the top row.
- *Rotate piece* (still under consideration).
- *Speed up falling by shifting* (still under consideration).

1.2.3 Board

A fixed-width grid consisting of 0s, 1s, or empty cells (spaces). Each row carries:

- a bit pattern, and
- a bitwise operator (**AND**, **OR**, **XOR**, **NAND**, ...).

1.2.4 Core Mechanic

When one or more falling bits touch one or more bits of a row, the following operation is applied cell-by-cell:

$$\text{row_bit} \leftarrow \text{piece_bit} \langle \text{OP} \rangle \text{row_bit}$$

Result	Outcome
1	Cell becomes filled (1)
0	Cell becomes filled (0) (.)

After the interaction, remaining bits continue falling and can trigger multiple row interactions in a single drop.

1.2.5 Clearing Rows / Columns

1. When all row bits become 0 cells.
2. When all row bits become 1 cells.

1.3. Difficulty Curve

The interval at which new rows spawn with random operations decreases over time, leaving the player less space and time to think. As the game progresses, more complex operations appear (e.g., **XOR**, **NAND**, **NOR**).

1.4. Technology Stack

Component	Details
Language	C++
Rendering	Console output (<code>stdout</code>) - board printed to terminal each tick, then cleared.
Input	Console input (<code>stdin</code>); <code>termios.h</code> for raw keypress reading without Enter (Linux).
Persistence	.txt file for saving high scores.

1.5. Activity Diagram

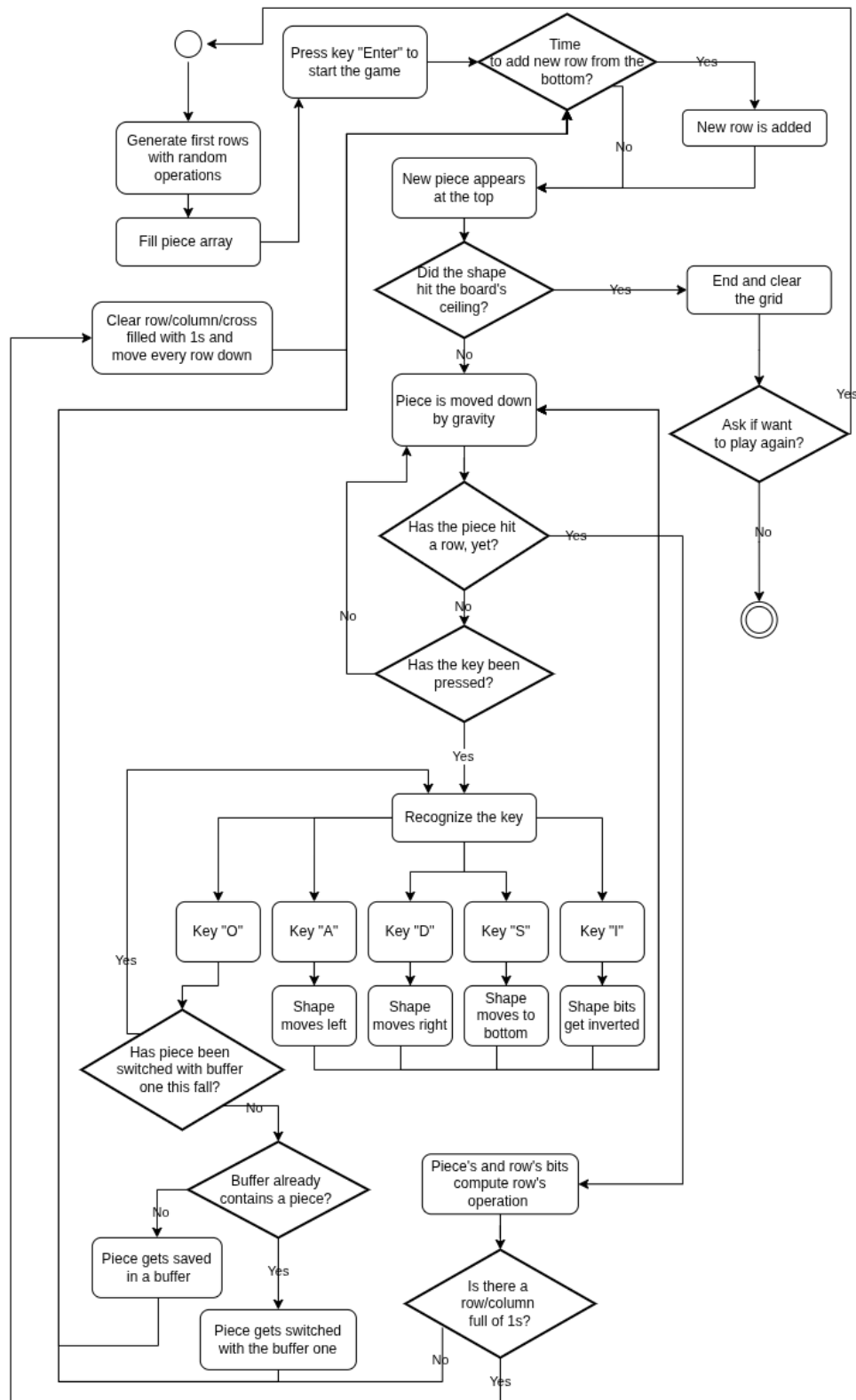


Figure 3: Activity Diagram

1.6. Data Flow Diagram

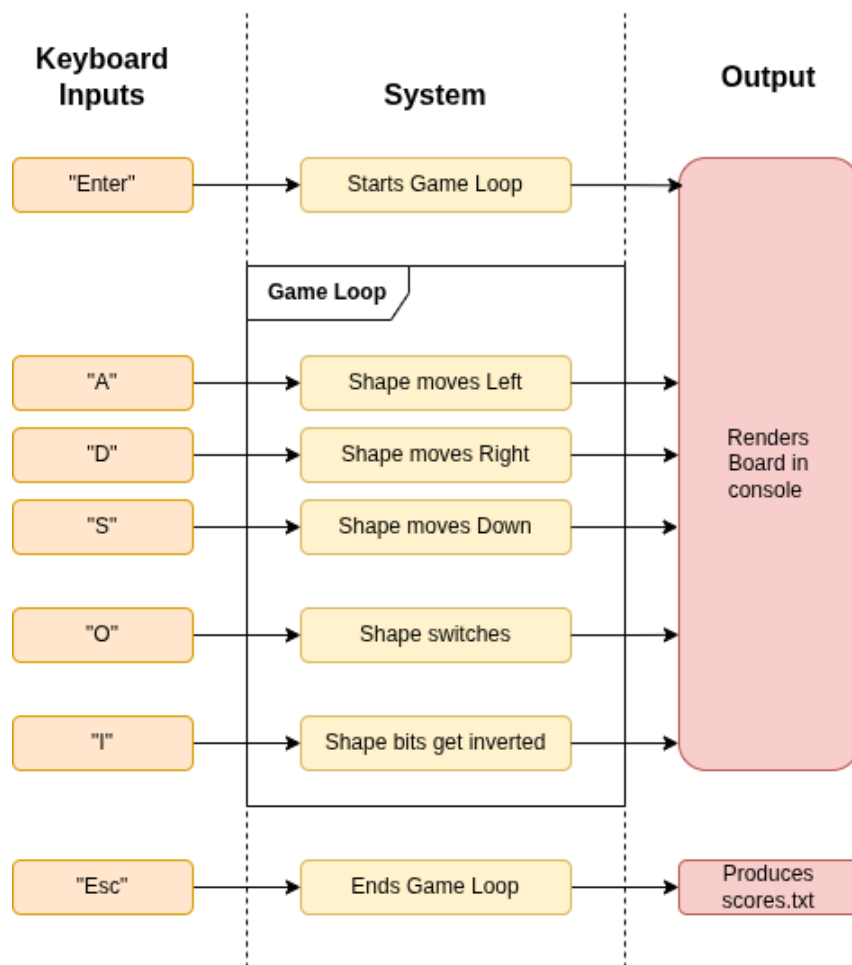


Figure 4: Data Flow Diagram

1.7. High-Level Architecture

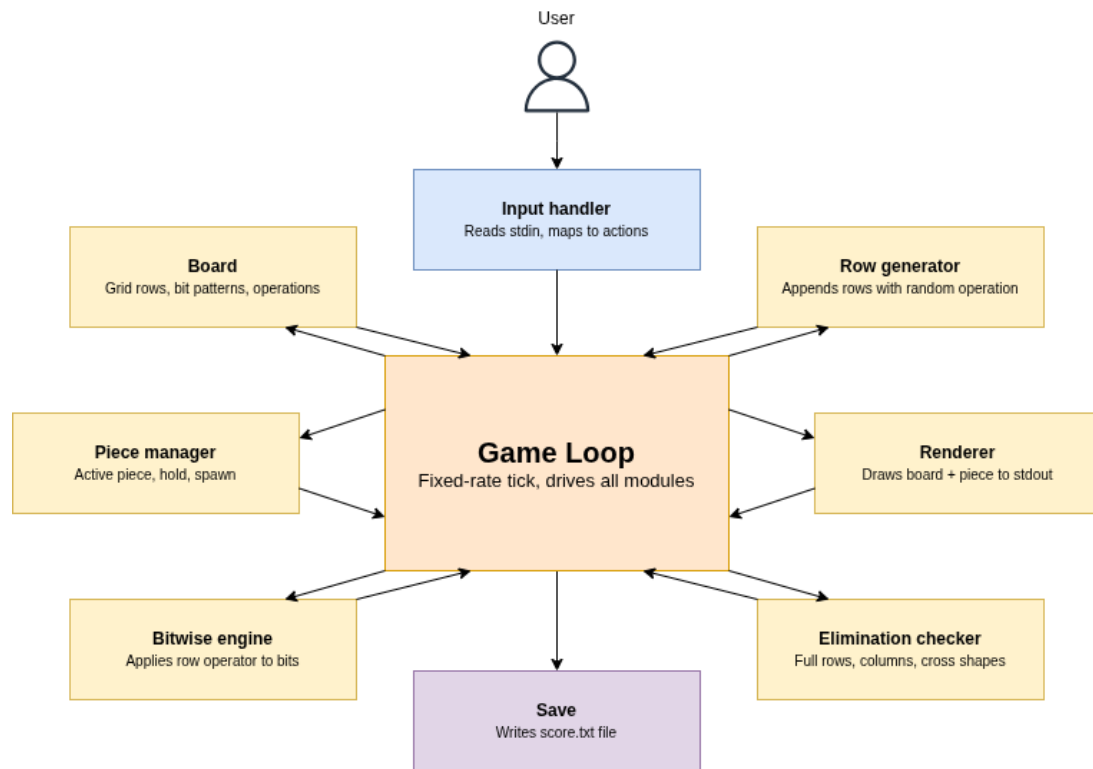


Figure 5: High Level Architecture Diagram

Module	Responsibility
Input Handler	Reads raw keypresses from <code>stdin</code> ; translates them into game actions (move, rotate, drop, hold).
Game Loop	The core tick cycle; drives all other modules at a fixed rate.
Piece Manager	Generates new pieces; manages the held piece; tracks the active piece's position and bit pattern.
Board	The grid of rows, each storing its bit pattern and assigned operator.
Bitwise Engine	Takes the piece bits and row bits, applies the row's operation, returns the result.
Elimination Checker	After each placement, checks for full rows, full columns, and cross (+) shapes; clears them.
Row Generator	Tracks when a new row is due; generates and appends it at the bottom with a random operation.
Renderer	Reads board and piece state; draws everything to <code>stdout</code> each tick.
Save	Writes the <code>.txt</code> file for scores.

1.8. UML diagrams

1.8.1 First Iteration

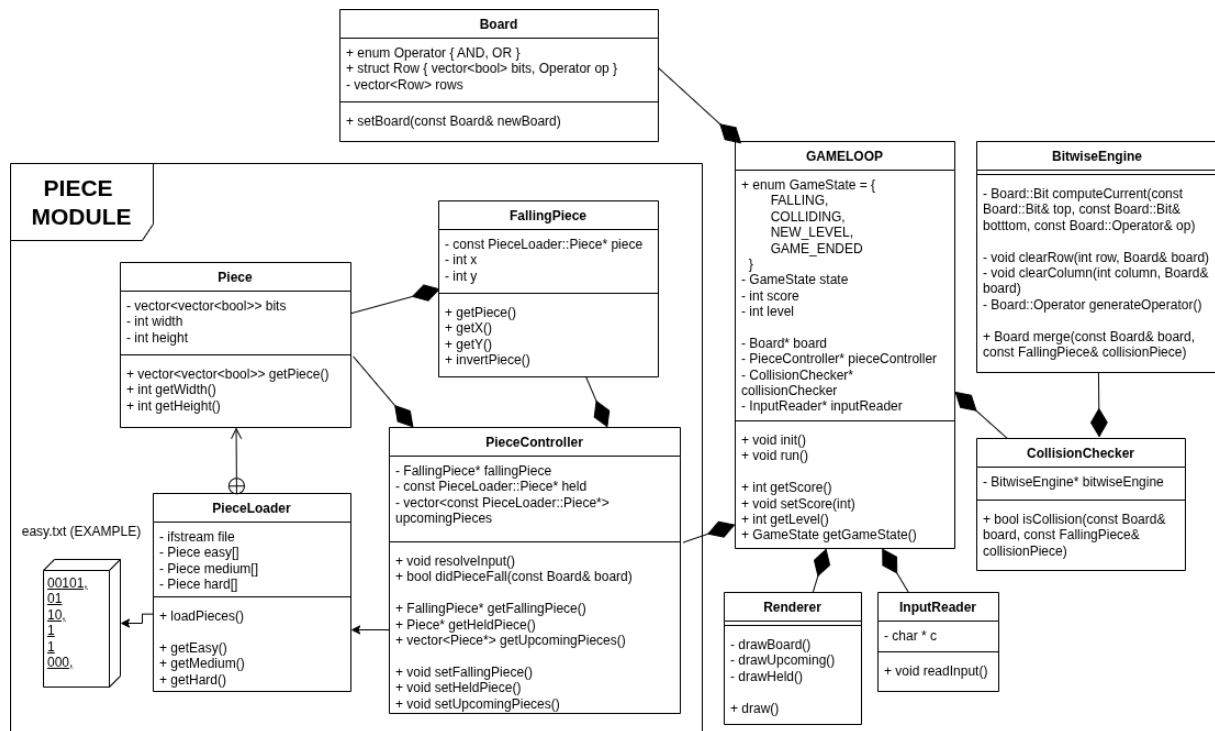


Figure 6: UML diagram to begin coding

1.8.2 Second Iteration - after TP2

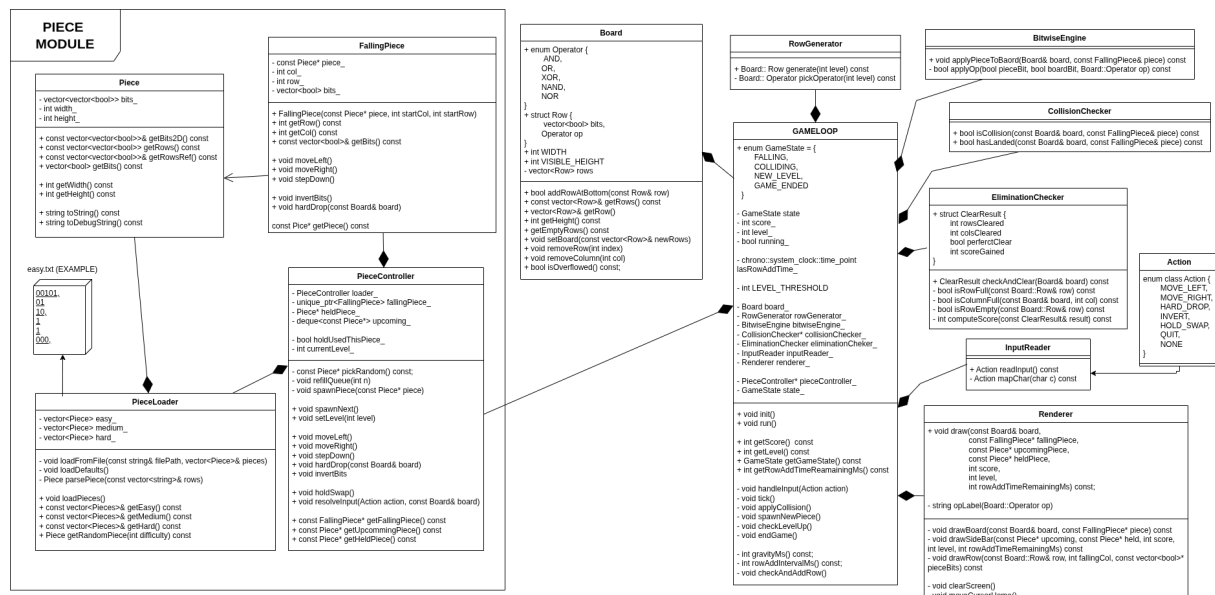


Figure 7: UML diagram after TP2

2. Plan for TP1 & TP2

TP2 - Core Game Loop

The goal of TP2 is a minimum playable version of Architetrix: a piece falls, interacts with the board via bitwise operations, and clears are detected. Every item below is required unless marked *nice to have*.

1. **Input handler.** Raw keypress reading via `termios.h`; translates keypresses into game actions: move left/right, hard drop.
2. **Piece gravity & tick cycle.** A fixed-rate game loop drives piece descent one row per tick. Hard drop (instant fall) is also handled here.
3. **Piece manager.** Loads piece definitions from a `.txt` file (separate easy / medium / hard sets). Selects a random piece, spawns it at the top of the board, and tracks its current position and bit pattern.
4. **Renderer.** Prints the full game screen to `stdout` each tick:
 - Bit rows with their assigned operator labels,
 - The currently falling piece,
 - Upcoming-piece bar,
 - Level and score fields.
5. **Bitwise engine - AND, OR.** Takes the falling piece's bit pattern and a row's bit pattern, applies the row's operator cell-by-cell, and writes the result back to the board.
6. **Elimination checker.** After each placement, scans the board for:
 - A row filled entirely with 1s (advanced clear),
 - A row filled entirely with 0s (basic clear)
 Cleared rows/columns are removed and the score is updated.
7. **Invert action (NOT).** Pressing `"i"` flips all bits of the currently falling piece (010 → 101).
8. **Piece buffer (hold / switch).** A one-slot buffer stores a piece. The player may swap the active piece with the buffered one; this exchange is permitted only once per new piece.
9. (*Nice to have*) **Difficulty-gated operators.** **XOR**, **NAND**, and **NOR** rows do not appear until the player reaches a defined level threshold, creating a gradual complexity curve.
10. (*Nice to have*) **Weighted piece selection.** As the level increases, harder pieces (from a harder set) become more likely to be selected, increasing difficulty organically over time.

TP3 - Polish & Features

TP3 extends the working game with the remaining operations, player tools, and persistence. Items marked *nice to have* are implemented only after the required tasks are complete.

1. **Extended bitwise engine.** Add **XOR**, **NAND**, **NOR** (and optionally **NOR**, **XNOR**) to the engine. (*Done by: Adrian*)
2. **Highscore** If Game Score surpasses Highscore it becomes new Highscore. (*Done by: Džiugas*)
3. **Score persistence.** Write the current score to a `.txt` file on game-over; read it back on launch to display the personal best. Update the file whenever a new high score is reached. (*Done by: Džiugas*)
4. (*Nice to have*) **Ghost Piece.** While a piece is falling, a ghost of the piece is displayed, showing where it would land if a hard drop were performed. This allows the player to

see exactly where the piece will be placed. (*Done by: Džiugas*)

5. (*Nice to have*) **Color encoding.** Assign distinct terminal colors to specific triggers: different operators highlighted per row, flash effect on clears, and visual distinction for the held piece. (*Done by: Adrian*)
6. (*Nice to have*) **Game Over screen.** (*Done by: Adrian*) Displays Reached score, Highscore and Reached Level. Game Over Screen has to be color coded.

3. File distribution for TP1 & TP2

Planned project structure and accountability distribution.

src/	
main.cpp	BOTH
game/	ADRIAN
GameLoop.h / .cpp	
GameState.h	
board/	ADRIAN
Board.h / .cpp	
RowGenerator.h / .cpp	
piece/	DŽIUGAS
Piece.h / .cpp	
FallingPiece.h / .cpp	
PieceLoader.h / .cpp	
PieceController.h / .cpp	
engine/	ADRIAN
BitwiseEngine.h / .cpp	
CollisionChecker.h / .cpp	
EliminationChecker.h / .cpp	
input/	BOTH
InputReader.h / .cpp	
render/	BOTH
Renderer.h / .cpp	
save/	DŽIUGAS
ScoreManager.h / .cpp	

4. Git Repository

Link to Git Repository: [Architetrix Git Repository](#)

5. Project Costs

Link to Cost Table: [Go to Google Sheets](#)

TP1	Builder	Architect	Exibitor	Designer	Critic	Facilitator	Meeting	Total	
Džiugas	02:00:00	00:00:00	00:00:00	00:00:00	00:00:00	00:00:00	10:00:00	14:20:00	
Adrian	00:00:00	00:45:00	00:00:00	00:00:00	00:00:00	00:00:00	10:00:00	16:15:00	
TP2	Builder	Architect	Exibitor	Designer	Critic	Facilitator	Meeting	Total	
Džiugas	12:08:00	06:00:00	00:00:00	00:00:00	00:00:00	00:00:00	04:35:00	22:43:00	
Adrian	08:50:00	04:55:00	01:00:00	00:00:00	00:00:00	03:55:00	04:35:00	22:15:00	
TP3	Builder	Architect	Exibitor	Designer	Critic	Facilitator	Meeting	Total	
Džiugas	03:45:00	00:00:00	00:00:00	00:00:00	00:00:00	00:00:00	00:40:00	04:25:00	
Adrian	03:45:00	00:00:00	00:45:00	00:00:00	00:00:00	00:00:00	00:40:00	5:10:00	
TOTAL	Builder	Architect	Exibitor	Designer	Critic	Facilitator	Meeting	Total	
Džiugas	17:53:00	06:00:00	00:00:00	00:00:00	00:00:00	00:00:00	15:15:00	17:28:00	plus +24
Adrian	12:35:00	05:40:00	01:45:00	00:00:00	00:00:00	03:55:00	15:15:00	19:40:00	plus +24

Figure 8: Project Costs

6. Demo Video TP3

Link to Demo Video: [Go to Demo Video](#)